Unsafezone

OFFENSIVE SECURITY

> learn
> build
> share
> hack



VULNERABILIDADES ANTIGAS QUE SE RECUSAM A MORRER

SQL Injection + IDOR/BOLA | tecnologias modernas, erros antigos

SQL Injection ainda importa em 2026?

> resposta_curta: SIM_

A pergunta real é: por que ainda encontramos falhas conhecidas há décadas?

> SQL INJECTION / HISTORY

Uma vulnerabilidade que atravessou gerações



Prepared statements são conhecidos há décadas. A falha continua aparecendo.

> CASE STUDY / SQLi

TalkTalk — quando uma falha antiga vira prejuízo

156.959

clientes afetados

£42M

custos excepcionais reportados

2015

Ataque documentado como SQL Injection.

Páginas legadas e controles insuficientes ajudaram a transformar uma falha conhecida em incidente real.

Mensagem: vulnerabilidade antiga não significa risco antigo.

> CASE STUDY / CURRENT

2025 — SQL Injection em um Web Application Firewall

FortiWeb

CVE-2025-25257

SQLi → RCE

Um produto criado para proteger aplicações web também pode carregar falhas clássicas.

> LIVE DEMO / SQLi

PoC #01 — SQL Injection

LIVE DEMO

Laboratório local • comportamento vulnerável → correção → reteste

> request

```
$ curl http://lab.local/api/products?id=1
200 OK  → produto #1

$ curl 'http://lab.local/api/products?id=1%27'
500 / comportamento inesperado
```

> server-side

```
SELECT * FROM products
WHERE id = '$id';

// correção
WHERE id = ?
prepared statement
```

> ROOT CAUSE / SQLi

O problema não é o SQL. É transformar dados em instruções.



Correção: parametrização + menor privilégio + testes + observabilidade

> TRANSITION

A WEB MUDOU.

Monólito → Web Services → Mobile → REST API → Cloud → Microservices

A confiança excessiva no cliente, não.

> IDOR / BOLA

Estou autenticado. Mas estou autorizado?

> request A

```
GET /api/faturas/1001
Authorization: Bearer TOKEN_PAULO

200 OK → Cliente: Paulo
```

> request B

```
GET /api/faturas/1002
Authorization: Bearer TOKEN_PAULO

200 OK → Cliente: Maria
```

AUTENTICADO ≠ AUTORIZADO

2025 — IDOR em uma plataforma de IAM

ZITADEL

Identity & Access Management

User A → API → Object B

IAM

Falha de autorização em endpoints administrativos: autenticação correta não garante autorização correta.

> LIVE DEMO / BOLA

PoC #02 — IDOR / BOLA

Mesmo usuário. Mesmo token. Outro objeto.

> 1001

```
$ TOKEN=token_paulo
$ curl -H "Authorization: Bearer $TOKEN" \
  http://lab.local/api/invoices/1001

{"id":1001,"customer":"Paulo"}
```

> 1002

```
$ curl -H "Authorization: Bearer $TOKEN" \
  http://lab.local/api/invoices/1002

{"id":1002,"customer":"Maria"}
same token_
```

> THE LINK

Duas falhas diferentes. Um padrão recorrente.

SQL INJECTION

confia na entrada

→ **CONFIANÇA INDEVIDA** ←

IDOR / BOLA

confia no identificador

O cliente controla dados. O servidor precisa controlar significado, escopo e autorização.

> MYTH BUSTING

“Mas usamos tecnologia moderna...”

TLS 1.3 | Kubernetes | Cloud | JWT | OAuth

MFA | WAF | EDR | SIEM | Zero Trust

LÓGICA INSEGURA CONTINUA INSEGURA.

> DEFENSE

Como finalmente matar essas vulnerabilidades

SQL INJECTION

- ✓ **Prepared statements**
- ✓ Queries parametrizadas
- ✓ Least privilege no banco
- ✓ SAST / DAST / code review
- ✓ Reteste após correção

IDOR / BOLA

- ✓ **Object-level authorization**
- ✓ RBAC / ABAC bem definidos
- ✓ Filtrar por owner/tenant
- ✓ Testes negativos de API
- ✓ Logging + detecção de abuso

O atacante não se importa se a vulnerabilidade tem 25 anos ou 25 dias.

Ele se importa se ela funciona.

> AUTENTIQUE • AUTORIZE • VALIDE • MONITORE • TESTE_

Q&A



Unsafezone

CYBERSECURITY COMMUNITY